
redroid-organico

Publicación automatizada sobre Android nativo en contenedores, con aislamiento de red verificado por cuenta.

Documento	Arquitectura y operación
Rama	<code>main</code>
Revisión	<code>791bb34</code> – <code>74 commits</code>
Fecha	26 de julio de 2026
Alcance	Sistema completo: cola, worker, driver Android, red, seguridad y operación

Contenido

- | | | | |
|---|--------------------------------|----|-------------------------------|
| 1 | Qué es y qué problema resuelve | 7 | Aislamiento de red por cuenta |
| 2 | Stack y decisiones de fondo | 8 | Seguridad |
| 3 | Arquitectura | 9 | Modos de falla y diagnóstico |
| 4 | Modelo de datos | 10 | Operación |
| 5 | Ciclo de vida de un job | 11 | Qué está verificado y qué no |
| 6 | El driver Android | 12 | Límites conocidos |

1. Qué es y qué problema resuelve

Una cola de publicación asíncrona que maneja aplicaciones Android reales corriendo en contenedores, una instancia descartable por trabajo, con la salida de red de cada cuenta forzada por un proxy propio y comprobada desde adentro del dispositivo antes de tocar la aplicación.

El problema de fondo no es publicar contenido: es publicarlo desde muchas cuentas sin que las plataformas las vinculen entre sí. Eso descarta las dos salidas obvias. Las APIs oficiales exigen una identidad de aplicación compartida y limitan qué se puede hacer; la emulación x86 deja huellas de traducción binaria que son una bandera roja por sí mismas.

La respuesta del sistema es manejar la aplicación como lo haría una persona —toques sobre la interfaz real— sobre Android nativo en ARM, y darle a cada cuenta dos cosas que no comparte con ninguna otra: un volumen de datos propio y una dirección de salida propia.

Los tres invariantes

Casi todas las decisiones del código se explican por uno de estos tres. Vale la pena leerlos antes que el resto del documento.

Invariante	Cómo se sostiene
Nunca publicar dos veces	<code>Job.idempotencyKey</code> con constraint único, el id de la fila como id de BullMQ, y una verificación de <code>COMPLETED</code> antes de publicar. Un doble click, un reintento de red y una re-entrega de la cola resuelven todos al mismo job.
Nunca reportar un éxito que no ocurrió	Los flujos de interfaz exigen al menos una aserción; el schema rechaza uno que no la tenga. El identificador de la publicación sale del texto que muestra la aplicación, no de un UUID generado antes de tocar nada.
Nunca salir por una dirección que no corresponde	El dispositivo corre dentro del namespace de red de su pasarela, con un firewall que rechaza todo lo demás, y cada corrida verifica desde adentro de Android por qué dirección sale. Un desvío es un job que falla, no una publicación desde la IP equivocada.

El tercero es el que más código costó y el que ocupa la sección 7 entera. Los otros dos son problemas conocidos con soluciones conocidas; ese no.

2. Stack y decisiones de fondo

Capa	Elección	Por qué esa y no otra
Panel y API	Next.js 14, React 18	App Router; las rutas de API y el panel comparten validación y tipos.
Datos	PostgreSQL 16 + Prisma 7	La base es la fuente de verdad del estado de cada job, no la memoria del worker. Eso es lo que hace correcto tener varias réplicas.
Cola	BullMQ 5 + Redis 7	Reintentos con backoff, jobs demorados y cola de descarte ya resueltos. <code>appendonly</code> activado: sin eso un reinicio pierde lo programado.
Android	ReDroid 13 en ARM	Android nativo en contenedor. En Graviton no hay traducción binaria, que es una de las señales que se busca evitar.
Automatización	Appium + UiAutomator2, ADB	Los flujos son datos —JSON validado por schema— no código. Cambiar un selector no exige un despliegue.
Aislamiento	tun2socks + namespaces de Docker	Ver sección 7. No se configura ningún proxy dentro de Android.
Validación	Zod 4	El mismo schema valida en el alta por CLI, en la API y en el worker. No pueden discrepar.

Dos decisiones que conviene entender

Los flujos de interfaz son datos, no código

Un flujo es una lista de pasos declarativos guardada en las credenciales de la cuenta. Acciones: `tap`, `type`, `assertVisible`, `assertGone`, `wait`.

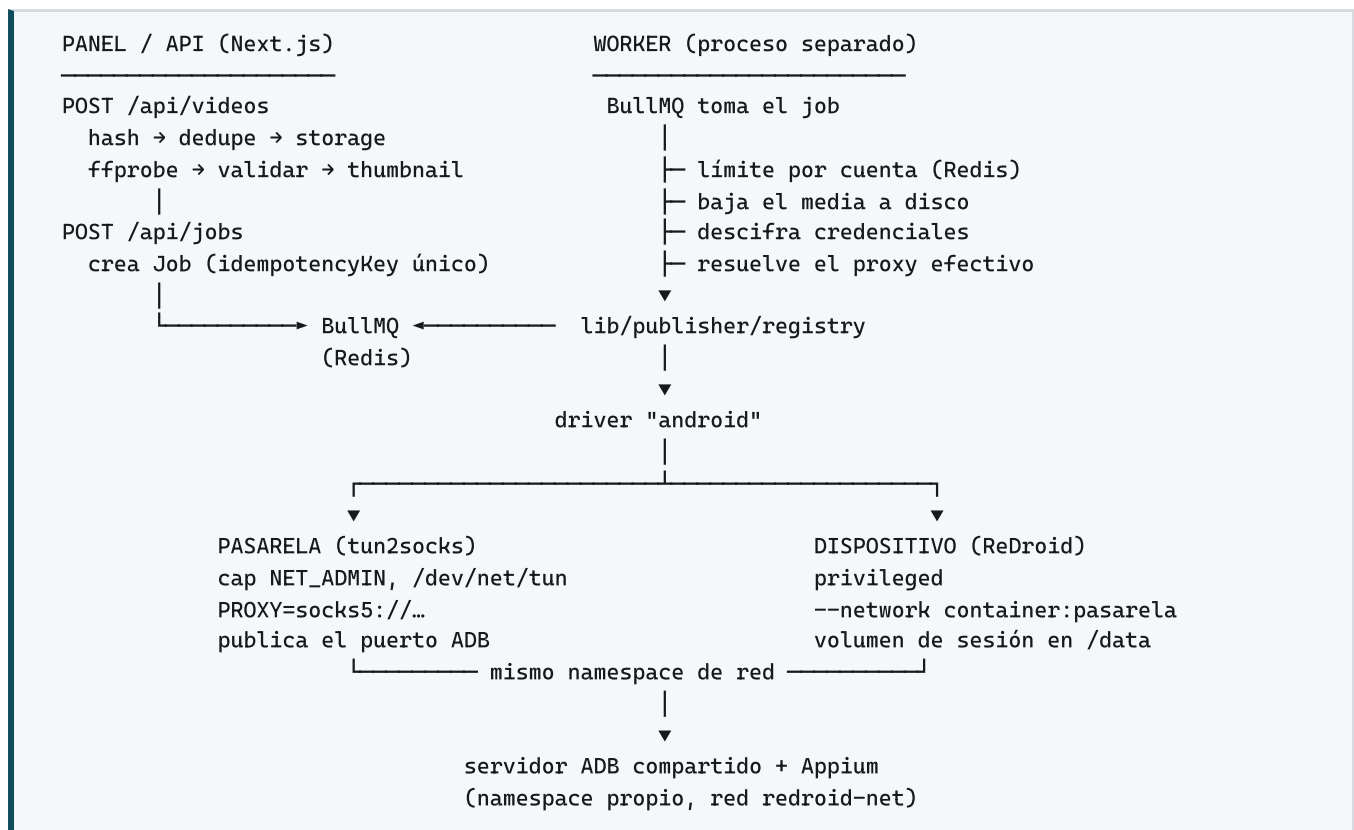
```
{
  "action": "assertVisible",
  "name": "el perfil muestra el usuario",
  "using": "id",
  "value": "com.target.app:id/profile_handle",
  "captureText": true,
  "timeoutMs": 15000
}
```

El schema **rechaza un flujo sin al menos una aserción**. Sin ella el run solo puede reportar éxito, que es peor que no tener nada: da la señal de que todo anda mientras la aplicación está rota. La tolerancia a interfaz condicional existe pero es explícita — `optional: true`, para un permiso de primer arranque— y todo lo demás falla fuerte.

Nada se configura dos veces

Las credenciales de una cuenta se validan contra `androidCredentialsSchema` en el momento del alta. Un flujo con una estrategia de localización inexistente, un puerto que no es número o una URL malformada se rechazan ahí, no adentro de un worker a las tres de la mañana. El mismo schema aplica los valores por defecto al leer, así que una cuenta guardada antes de que existiera una opción la recibe igual.

3. Arquitectura



Los contenedores del par pasarela/dispositivo se crean y destruyen por cada job; el resto es permanente.

Separación de procesos

El panel y el worker son dos procesos independientes que solo se comunican por PostgreSQL y Redis. El panel nunca ejecuta un job: responde inmediatamente con una fila creada, y el worker la toma cuando puede. Eso es lo que evita que una publicación de tres minutos se convierta en un timeout de HTTP.

El worker necesita el socket de Docker porque crea y destruye contenedores. Eso equivale a root en el host: aceptable para este despliegue, no para uno compartido.

Dónde está cada cosa

Ruta	Contenido
app/api/	Rutas HTTP: jobs, videos, cuentas, proxies, auth, salud, stream SSE
lib/android/	ADB, Appium, flujos, contenedor efímero, pasarela, política de egress, verificación, reaper
lib/proxy/	Validación de proxies, armado de la URL, asignación por cuenta
lib/publisher/	Contrato del adaptador, errores clasificados, drivers
lib/worker/	Procesador de jobs, limitador por cuenta, arranque del worker
lib/crypto/	Cifrado AES-256-GCM de credenciales, con rotación
scripts/	Alta de cuentas, export/import, diagnóstico de pasarela y de egress, carga, reportes
tests/integration/	15 archivos contra Postgres y Redis reales

4. Modelo de datos

Ocho modelos. Los cuatro primeros son el núcleo.

Modelo	Rol
Account	Una identidad en la plataforma objetivo. Guarda sus credenciales cifradas —que incluyen qué aplicación manejar y con qué flujos—, su estado de sesión, sus límites de concurrencia y a qué proxy sale.
Job	Una corrida. Publicación o vinculación interactiva. Lleva su clave de idempotencia, su presupuesto de reintentos, el endpoint del dispositivo mientras espera a una persona, y las métricas de la corrida.
Proxy	Una salida de red. Pertenece a un usuario y puede compartirse entre cuentas —el panel avisa cuándo—. La contraseña va cifrada con la misma clave que el resto de las credenciales.
Video	Media validada: duración, dimensiones, códec, checksum para deduplicar reenvíos.
JobLog	Bitácora por job, redactada por lista de claves. Es lo que muestra el panel.
User, Session, ApiToken	Operadores, sus sesiones por cookie y los tokens para acceso servidor a servidor.

Dos estados que no son el mismo

`Account.status` es *intención del operador*: `ACTIVE` significa "quiero que se use". `Account.sessionState` es *realidad del dispositivo*: `VERIFIED` significa "alguien se logueó y un flujo lo confirmó". Una cuenta puede estar activa y sin sesión. Colapsarlos haría indistinguible "habilitada" de "usable".

Estados de un job

Estado	Significa
QUEUED / SCHEDULED	Esperando worker, o esperando su hora.
PROCESSING	Un worker lo tomó.
AWAITING_HUMAN	Hay un dispositivo vivo y una persona logueándose en él. No hay nada mal en un job que pasa diez minutos acá.
VERIFYING	La persona dijo que terminó y el worker está comprobando si la aplicación coincide. Separado de lo anterior: un job que muere acá tiene que distinguirse de uno que murió esperando.
COMPLETED / FAILED	Terminal.
CANCELLED / DEAD	Cancelado por el operador; o agotó los reintentos y quedó en la cola de descarte.

Por qué importa `AWAITING_HUMAN`. El barredor de contenedores huérfanos decide contra la base. Antes trataba todo lo que no fuera `PROCESSING` como basura, y destruía el dispositivo de una vinculación a los noventa segundos, en la mitad del login de la persona.

5. Ciclo de vida de un job

Idempotencia

El cliente manda una cabecera `Idempotency-Key`; la columna tiene constraint único. Dos requests concurrentes con la misma clave resuelven al mismo job: el que pierde la carrera lee la fila que escribió el que ganó, en vez de devolver un conflicto. El id de BullMQ es el id de la fila, así que un re-encolado accidental lo descarta Redis. Y antes de publicar se verifica `COMPLETED`, por si la cola entrega dos veces.

Reintentos, y qué no se reintent

Backoff exponencial: 15s, 60s, 240s. Lo que decide si hay reintento es la clasificación del error, no el tipo de excepción.

Situación	Reintenta	Por qué
Dispositivo sin bootear, Appium inalcanzable, push incompleto	Sí	Infraestructura que todavía no está lista.
Un selector que nunca aparece	No	Tres reintentos gastan cinco minutos, informan lo mismo y esconden un defecto real detrás de "intento 3/3".
La aplicación se cae al arrancar	No	Es un hallazgo, no una intermitencia.
Credenciales o flujo inválidos	No	Reintentar no arregla una configuración.
El dispositivo sale por la IP del host	No	Cada reintento sería otra oportunidad de publicar desde la dirección equivocada.

Límite por cuenta

Dos límites independientes en Redis, compartidos entre réplicas: `maxConcurrent` y `minIntervalSeconds`. Cuando frenan un job, este se posterga **sin consumir un intento**: el throttling nunca agota el presupuesto de reintentos. La misma reserva la toman las publicaciones y las vinculaciones, porque las dos montan el volumen de sesión de la cuenta y dos a la vez lo corromperían.

Recuperación

Un worker que muere a mitad de un job dejaba la fila en `PROCESSING` para siempre. Al arrancar, `recoverOrphans()` busca esas filas, verifica contra BullMQ que nadie las esté procesando, y las re-encola. El apagado con SIGTERM espera a que terminen los jobs activos en vez de matarlos: una publicación interrumpida y re-entregada puede ser un post duplicado.

Vinculación interactiva

Es el flujo inverso al de publicación: el worker levanta un dispositivo, pone la aplicación en pantalla, publica el endpoint del visor y *se corre del medio*. La persona se loguea mirando la pantalla real en el panel. Cuando dice que terminó, recién ahí el worker corre el flujo de verificación.

Marcar una cuenta como verificada porque alguien apretó un botón sería la misma categoría de mentira que un identificador de publicación inventado. Por eso `verifyFlow` es obligatorio para poder usar vinculación interactiva, y una confirmación que la aplicación no respalda deja la cuenta sin verificar.

Un dispositivo vivo cuesta. Una pestaña cerrada no avisa nada, así que el deadline es lo único que termina una sesión abandonada. Los jobs interactivos se crean con un solo intento: un reintento levantaría un segundo dispositivo que nadie está mirando.

6. El driver Android

El driver no sabe de dónde sale el dispositivo. Eso lo decide un *provider*: uno para un dispositivo que ya existe y sobrevive al job —un teléfono por USB, un emulador local— y otro que crea un contenedor descartable por corrida. El segundo es el que importa.

Ciclo de vida del contenedor efímero

```
acquire()
├─ rechaza si la cuenta ya tiene un contenedor vivo    (dos jobs = un volumen = corrupción)
├─ crea el volumen de sesión de la cuenta
├─ levanta la pasarela y le conecta la red de control
├─ docker run --privileged --network container:pasarela
├─ espera el puerto ADB, conecta, espera el boot
├─ fija el namespace al túnel y verifica la salida      ← sección 7
└─ verifica que la aplicación esté instalada
release()
├─ adb disconnect    (sin esto el servidor acumula entradas "offline")
├─ borra el dispositivo
└─ borra la pasarela  (en ese orden, obligatorio)
```

Tres compuertas de readiness

Cada una pasa mientras la siguiente falla, y por eso están las tres: el contenedor corre, ADB acepta conexión, y Android terminó de bootear. ReDroid marca `sys.boot_completed` mientras la animación de arranque sigue y el gestor de paquetes se está acomodando; entregar ese dispositivo a Appium produce corridas que fallan con "elemento no encontrado" sin que el flujo tenga nada malo.

Hay una cuarta, específica del almacenamiento: `/sdcard` es un montaje FUSE que se prepara después del boot. Empujar un archivo en esa ventana transfiere los bytes y después falla al crear el archivo, en un directorio que funciona perfecto un minuto más tarde. Se prueba la capacidad —¿se puede crear un archivo acá?— en vez de una propiedad que correlacione con ella.

El volumen de sesión

Uno por cuenta, montado en `/data`, que es donde Android guarda todo lo que debe sobrevivir entre corridas: el login, las bases de la aplicación, las preferencias. Dos cuentas nunca se ven el estado.

Ese montaje explica un detalle que confunde: la aplicación bajo prueba **no se puede hornear en la imagen**. Las aplicaciones de usuario viven bajo `/data`, que ReDroid monta en tiempo de ejecución, y `docker commit` solo captura la capa de escritura del contenedor. La imagen se construye, se verifica y sale vacía. Por eso el APK se instala en la primera corrida de cada cuenta y persiste en su volumen.

Que no queden zombies

Un contenedor Android sobrevive al proceso que lo creó y se come un gigabyte de RAM. Tres mecanismos independientes, porque cualquiera se puede saltar:

1. El **finally** del publisher. El camino normal. Corre aunque Appium explote a mitad del flujo, y aunque la adquisición falle a mitad de camino.

2. **Timeout en cada llamada a Docker.** Un demonio trabado no puede convertir "destruí el contenedor" en una promesa que nunca resuelve.

3. **El barredor.** Lo único que sirve cuando el worker recibe SIGKILL y ningún `finally` llega a correr.

El barredor decide contra la base de datos, no contra estado en memoria: un contenedor cuya etiqueta de job apunta a uno que no está vivo es basura sin importar qué worker lo creó. Eso lo hace correcto con varias réplicas. Ante la duda no mata: uno más joven que el período de gracia se deja en paz, y si no puede consultar el job lo reporta y sigue.

Barre en un orden que no es preferencia sino obligación: primero los dispositivos, después las pasarelas. Docker se niega a borrar un contenedor mientras otro le presta el namespace de red, así que barrer en el orden de la lista dejaría todas las pasarelas atrás, para reintentar en cada barrida para siempre.

7. Aislamiento de red por cuenta

Esta sección documenta el problema más difícil del sistema y su solución. Se escribió después de resolverlo contra una máquina real, y varios de los hallazgos contradicen lo que parecía obvio al empezar.

7.1 El patrón

No se configura **ningún** proxy dentro de Android. El worker levanta un contenedor `tun2socks` por job y arranca el ReDroid dentro de `su` namespace de red:

```
docker run --cap-add NET_ADMIN --device /dev/net/tun \
  --network redroid-net -p 127.0.0.1::5555 \
  -e PROXY=socks5://usuario:clave@pasarela:puerto \
  --name redroid-gw-<jobId> xjasonlyu/tun2socks:v2.5.1

docker run --privileged --network container:redroid-gw-<jobId> \
  --name redroid-job-<jobId> <imagen golden>
```

La razón de fondo: el dispositivo no tiene ninguna interfaz propia. No hay una opción que una aplicación pueda ignorar, ni tráfico UDP que se escape del proxy del sistema, ni estado que se pierda al restaurar la sesión desde el volumen. La única salida es el `tun0` de la pasarela.

Consecuencias estructurales:

- **Los puertos son de la pasarela.** Un contenedor que comparte namespace no puede publicar nada, así que el 5555 de ADB se publica ahí y el serial del dispositivo sale de la pasarela.
- **El teardown tiene un orden obligatorio,** por la misma razón.
- **Una cuenta con proxy exige contenedor efímero.** Sobre un dispositivo que el sistema no creó no hay namespace del cual apropiarse, así que el job falla en vez de publicar sin máscara.

7.2 Por qué compartir el namespace no alcanza

Acá es donde el diseño ingenuo se rompe. `netd`, el demonio de red de Android, no es un inquilino pasivo del namespace: instala sus propias reglas de ruteo por política y le estampa a cada socket el identificador de red con `SO_MARK`. El resultado es que el tráfico de Android matchea *sus* reglas y sale por `eth0`, mientras un `curl` sin marca, en el mismo namespace, sigue obedientemente el túnel.

Nada se escapa del namespace —no puede—: `netd` simplemente gana por prioridad. Esto es lo que se ve dentro de un dispositivo real:

```

0:      from all lookup local
90:     from all fwmark 0x22b lookup 1002      ← nuestras
95:     from all to 172.19.0.0/16 lookup 1002
100:    from all lookup 555

----- de acá abajo, netd
10000: from all fwmark 0xc0000/0xd0000 lookup 99
11000: from all iif lo oif eth0 uidrange 0-0 lookup 1002
16000: from all fwmark 0x10064/0x1ffff iif lo lookup 1002
17000: from all iif lo oif eth0 lookup 1002
18000: from all fwmark 0/0x10000 lookup 99
23000: from all fwmark 0x64/0x1ffff iif lo lookup 1002
31000: from all fwmark 0/0xffff iif lo lookup 1002
32000: from all unreachable

```

Once reglas, todas en la banda 10000–32000. La de 23000 es la que se llevaba el tráfico por fuera del proxy. El diagnóstico se hace en dos comandos:

```

docker exec <pasarela> ip route get 1.1.1.1          # sin marca: lo que hace curl
docker exec <pasarela> ip route get 1.1.1.1 mark 100 # con netId: lo que hace Android

```

7.3 Las tres capas

Se aplican por `docker exec` después de que Android terminó de bootear, no en el `docker run`: `netd` inserta sus propias reglas mientras arranca, y lo que se escriba antes queda detrás.

Capa	Qué hace	Si falta
1. Ruteo	Las mismas reglas en prioridad 90 y 100, por debajo de todo lo de <code>netd</code> . La 100 no tiene selector: <code>matchea</code> todo paquete sin importar la marca.	No hay aislamiento. Es la capa que sostiene todo.
2. Marcado	<code>mangle OUTPUT</code> borra la marca de <code>netd</code> , lo que además obliga al kernel a rehacer el lookup de ruta.	Redundante con la 1. Se saltea con un aviso.
3. Firewall	Decida lo que decida el ruteo, solo pueden emitir: <code>lo</code> , el túnel, las respuestas entrantes, la salida de la pasarela hacia su proxy y la subred de control. El resto, <code>REJECT</code> .	Un desvío vuelve a ser una IP equivocada en vez de una conexión que falla.

La red de control existe para que la única excepción del firewall no sea un agujero: se declara `internal`, o sea que Docker no le da ruta fuera del host. Esa restricción vive *afuera* del namespace, donde Android no la alcanza ni siendo privilegiado.

7.4 Cuatro hallazgos que costaron una corrida cada uno

Todos se descubrieron contra la máquina real. Se documentan porque ninguno era evidente y todos vuelven a aparecer en cualquier host nuevo.

El backend de iptables

La imagen de la pasarela apunta `/sbin/iptables` al binario *legacy*. En un host con `nftables` —Ubuntu 24.04 en adelante— eso contesta `Table does not exist`: primero para `mangle`, después para `filter`, y no se termina más. Perseguirlo cargando módulos `legacy` de a uno es emular a mano un backend que el kernel ya tiene. El script prueba `iptables-nft`, `iptables-legacy` e `iptables`, y se queda con el primero que conteste.

La tabla del bypass no es `main`

La pasarela necesita alcanzar su propio proxy sin pasar por el túnel que ella misma sirve. La regla obvia apunta a `main`, y en Android `main` está vacía: `netd` se lleva la interfaz física a una tabla propia. El paquete no encontraba nada, atravesaba las once reglas de `netd` sin `match`ear ninguna —la única `catch-all` exige marca cero— y moría en `32000: unreachable`. La pasarela perdía la conexión a su proxy y el síntoma era un túnel que no relayaba nada. La tabla ahora se descubre: la primera ruta por defecto que no sea la del túnel.

Las subredes directas se llevaban a ADB

La tabla del túnel tiene *solo* las rutas del túnel: `tun2socks` no copia las que había. Así que la regla `catch-all` mandaba al túnel también lo destinado a la subred local, que es donde vive la conexión ADB. El plano de control se cortaba y todas las sondas leían como "sin red" —el mismo síntoma que el problema original—. Las subredes directamente conectadas ahora salen antes del `catch-all`.

El proxy puede rechazar el destino

Un endpoint elegido por IP para esquivar el DNS resultó ser un resolver público, y los proveedores residenciales los bloquean de rutina para evitar DNS tunneling: la respuesta fue `connection not allowed by ruleset` y la lectura ingenua acusaba a un proxy que funcionaba. La verificación ahora usa un host común y el **worker resuelve el nombre**, pasándole la dirección a `curl --resolve`: sin DNS en el dispositivo y sin una IP de resolver en ninguna lista negra.

DNS sigue siendo un tema aparte. La verificación lo esquiva a propósito, pero la aplicación bajo prueba no puede. DNS es UDP y un SOCKS5 sin UDP ASSOCIATE lo descarta en silencio. La salida limpia es forzar DNS sobre TCP con Private DNS (DoT, puerto 853), que atraviesa el túnel sin depender de UDP.

7.5 La verificación

Todo lo anterior describe *lo que el worker pidió*. Una sola cosa dice *lo que Android hizo*: preguntarle al dispositivo, desde adentro, por qué dirección sale.

```
[info] Egress verified from inside the device  deviceIp=203.0.113.7 directIp=190.x.x.x
```

La comparación es contra la dirección **del host**, no la del proxy. Un residencial rotativo entrega una salida distinta por conexión, así que comparar contra el proxy daría falsos positivos todo el día; comparar contra la IP que se filtraría es inequívoco. Si coinciden, el job falla con `egress_leak_detected`, permanente, y el contenedor se destruye antes de que el flujo toque la aplicación.

Una imagen AOSP no trae ningún cliente HTTP —ni `curl`, ni un toybox con el applet `wget`—, así que el operador apunta `probeBinary` a un `curl` estático y el worker lo copia a `/data/local/tmp` en la primera corrida de cada cuenta. Persiste en el volumen de sesión, igual que el APK y por el mismo motivo.

Cuando la verificación falla y no contesta ni el dispositivo ni la pasarela, el error se lleva puesto el estado del namespace: reglas de ruteo, todas las tablas y **los contadores del firewall**. Esos contadores son los que separan "lo bloqueó la política" de "el proxy está caído", y los contenedores se destruyen en un `finally`: lo que no se capture ahí no existe más.

8. Seguridad

Credenciales

`Account.credentials` y `Proxy.password` se guardan cifrados con AES-256-GCM en un sobre `iv || tag || ciphertext`. Se descifran solo dentro del worker y solo durante un job. La API nunca los devuelve: el panel recibe un booleano que dice si hay contraseña, no la contraseña.

Rotación sin ventana de migración

`CREDENTIALS_KEYS_OLD` acepta claves retiradas separadas por coma, usadas **solo para descifrar**. Como GCM autentica, probar varias es seguro: una clave incorrecta falla limpio en vez de devolver basura. Se pone la nueva como primaria, se mueve la vieja al respaldo, y un script re-sella las filas. Es idempotente y **nunca toca una fila que no pudo abrir**: reporta cuáles fallaron y sale con código de error, porque perder el dato es peor que un reintento con la clave correcta.

Redacción en los logs

El registrador de jobs redacta por lista de claves antes de escribir en la base. Cubre credenciales, tokens, contraseñas, cookies y configuraciones de proxy. La forma deliberada y ya redactada se registra bajo otro nombre — `egress` — justamente para que la clave `proxy` se pueda blanquear sin excepciones.

Un detalle de la misma familia: las credenciales se percent-encodean al armar la URL del proxy. Sin eso, una contraseña con un `@` mueve el límite del host y la URL apunta a otro lado.

Acceso

Sesiones con cookie `httpOnly`; en la base se guarda solo el hash SHA-256 del token, así que un volcado de esa tabla no se puede reproducir como login. Contraseñas con `bcrypt`. El registro público está apagado por defecto. Un cambio de contraseña invalida *todas* las sesiones incluida la que lo pidió: es la respuesta estándar a una sospecha de compromiso, y dejar las otras vivas anularía el sentido.

9. Modos de falla y diagnóstico

Los errores están clasificados en el código con un identificador estable. La tabla siguiente es la referencia operativa.

Código	Reintenta	Qué mirar
device_not_ready	Sí	Logs del contenedor. Casi siempre binderfs o memfd en el host.
app_not_installed	No	El <code>packageName</code> no coincide con lo que declara el APK.
app_not_running	No	La aplicación se cayó al arrancar. Es un hallazgo.
ui_step_not_found	No	El selector no existe. <code>dump-ui.sh --artifact</code> muestra qué había en pantalla cuando falló.
appium_unreachable	Sí	El servicio de Appium.
invalid_android_credentials	No	El mensaje dice qué campo.
proxy_requires_ephemeral_device	No	La cuenta tiene proxy pero no un bloque <code>redroid</code> .
egress_unreachable	Sí	Distingue solo dos casos: si todo dio código 127, falta el cliente HTTP en la imagen; si todo dio timeout, el túnel se traga los paquetes.
egress_leak_detected	No	El dispositivo salió por la IP del host. Nunca se reintenta.

Herramientas de diagnóstico

Comando	Para qué
<code>npm run gateway:test -- --proxy <l></code>	Reproduce la ruta de salida en aislamiento. Bisecta credenciales contra ruteo, y con <code>--as socks5</code> prueba el mismo endpoint como otro tipo de proxy.
<code>npm run device:egress -- --account <id></code>	La salida real del dispositivo. <code>--hold</code> lo deja arriba para mirarlo por el visor.
<code>./scripts/dump-ui.sh</code>	Los selectores que la aplicación expone de verdad. Con <code>--artifact</code> , sobre el volcado que guardó una corrida que falló.
<code>npm run account:list</code>	Cuentas con su configuración descifrada y redactada, y por qué salida sale cada una.
<code>tsx scripts/lastJob.ts</code>	El último job con su estado, su error y sus logs.

El mensaje genérico del panel nunca dice el motivo real. Está en el log del job o en el del worker. Los errores del driver Android incluyen las últimas líneas del contenedor: se leen *antes* de destruirlo, porque hacerlo después reporta "No such container" en vez de la causa — el diagnóstico destruido por la limpieza que debía explicarlo.

10. Operación

Despliegue

Dos servicios de systemd, `redroid-web` y `redroid-worker`, corriendo sobre el host. No hay paso de compilación: el worker corre bajo `tsx` y el panel bajo Next, y los dos transpilan al vuelo. Actualizar es casi siempre reiniciar el servicio que usa el archivo que se tocó.

```
git pull
npm ci                # si cambió package-lock
npx prisma migrate deploy # si hay migraciones nuevas
sudo systemctl restart redroid-worker redroid-web
```

Los datastores y el stack de Android van en contenedores: `docker compose --profile android up -d` levanta Postgres, Redis, el servidor ADB compartido, Appium y el visor de pantalla.

Un cambio de credenciales no necesita reinicio. El worker las lee de la base en cada job. El reinicio es para cambios de código.

Preparar el host

Verificado sobre Ubuntu en Graviton (ARM64). Nada de esto es opcional:

Requisito	Por qué
binderfs montado	Los kernels modernos no crean <code>/dev/binder</code> ; los dispositivos viven en binderfs. Sin eso el contenedor no arranca.
memfd, no ashmem	Si el kernel no tiene ashmem, ReDroid 11 no arranca: hay que usar imagen 12 o 13 y pedir memfd explícitamente. Sin eso el contenedor arranca y se cuelga a mitad del boot sin error visible, que es la peor forma de fallar.
<code>/dev/net/tun</code>	Solo para proxies: la pasarela crea su dispositivo túnel desde adentro del contenedor.

Los módulos legacy de netfilter **no** hacen falta: el script elige el backend que el kernel conteste. Ver 7.4.

Alta y edición de cuentas

```
npm run account:list
npm run account:export -- --account <id> --out cuenta.json
# editar
npm run account:add -- --account <id> --driver android --credentials-file cuenta.json
```

El import valida contra el mismo schema que usa el worker y, si algo está mal, **no escribe nada**. Es un reemplazo, no una fusión: lo que se borre del archivo se pierde. Por eso existe el export — antes, cambiar un campo exigía recordar la configuración entera, flujos incluidos.

11. Qué está verificado y qué no

Distinguir lo comprobado de lo escrito es parte del diseño de este sistema, y esta sección se mantiene con el mismo criterio.

Comprobado contra infraestructura real

- Ciclo completo de publicación: alta de media, validación con `ffprobe`, `thumbnail`, encolado, `worker`, publicación y logs.
- Idempotencia, reintentos con `backoff`, agotamiento hacia la cola de descarte, recuperación de huérfanos tras caída del `worker`, límite por cuenta.
- Autenticación completa: rechazo sin sesión, `throttling` de logins fallidos, cambio de contraseña con invalidación.
- Rotación de claves con re-sellado.
- El stack completo en contenedores, y el workflow de integración continua.
- **La cadena de aislamiento de red, de punta a punta contra un VPS real:** elección de backend de `iptables`, `bypass` de la pasarela hacia su proxy, ADB fuera del túnel, firewall sin un solo descarte espurio, y la verificación de salida contestando desde adentro de Android.

No verificado

- **Los flujos de interfaz contra la aplicación objetivo.** La mecánica del driver está cubierta por tests que hablan HTTP real contra un Appium simulado —incluida la garantía de que un flujo que no encuentra sus elementos falla en vez de reportar éxito—, pero los selectores concretos dependen de la aplicación y se escriben con `dump-ui.sh`.
- **El despliegue con HTTPS.** La cookie se marca `secure` sola en producción, pero eso requiere TLS delante.
- **Volumen sostenido.** Existen el encolador de carga y el reporte por región, pero no hay todavía una corrida larga que respalde números.

Cobertura

197 tests de integración contra Postgres y Redis reales, más 9 de humo que no requieren infraestructura. Los de la cadena de red usan un doble de Docker que reproduce la regla que importa: un contenedor cuyo namespace otro está usando no se puede borrar. Y el script de política de red se validó ejecutándolo contra la imagen real de la pasarela, incluyendo kernels simulados sin tabla `mangle`, sin `xt_mark` y sin backend legacy.

12. Límites conocidos

Límite	Detalle y salida
Proxies HTTP	Relayan solo TCP, y en la práctica ni eso: muchos resetean cualquier CONNECT que no vaya al 443, y tun2socks necesita TCP arbitrario. Si el proveedor ofrece las dos sobre el mismo endpoint, SOCKS5 siempre.
DNS dentro de Android	Es UDP y no sobrevive a un SOCKS5 sin UDP ASSOCIATE. La verificación lo esquivo; la aplicación no puede. Salida: Private DNS sobre TCP.
El socket de Docker en el worker	Equivala a root en el host. Aceptable en un despliegue dedicado, no en uno compartido.
Un proxy compartido entre cuentas	Está permitido y el panel lo avisa, pero dos cuentas en una misma IP residencial son exactamente la correlación que el sistema existe para evitar.
Reconfiguración en caliente	Cambiar el proxy de una cuenta aplica al próximo job. Mover un contenedor vivo a otro namespace de red no es algo que Docker sepa hacer.
La capa 2 del aislamiento	Depende de la tabla <code>mangle</code> y del target <code>MARK</code> . Donde el kernel no los tenga se saltea con un aviso; es defensa en profundidad, no el mecanismo.

Trabajo natural a continuación

1. Escribir los flujos reales de la aplicación objetivo con `dump-ui.sh`, empezando por `verifyFlow`.
2. Resolver DNS dentro de Android con Private DNS sobre TCP.
3. Alinear `scripts/testGateway.ts` con la política de red del runtime: hoy levanta la pasarela sin aplicarla, así que su veredicto y el del worker pueden divergir.
4. Una corrida de volumen sostenido que respalde el reporte por región con números propios.

Documento generado desde el repositorio en la revisión indicada en la portada. Las secciones 7 y 9 reflejan hallazgos obtenidos contra la máquina de producción y son las que conviene releer antes de tocar cualquier host nuevo.